Image Carousel: Machine Coding Walkthrough

Problem: Build an image carousel with previous / next buttons, dot indicators and autoplay.
Tech stack: React (Vite, JavaScript) + Tailwind CSS, `react-icons` (already in the project)
Time limit: 60 minutes

PART A: Thought process

Every machine coding round follows the same loop. Do it in this order and say each step out loud.

1. **Clarify requirements**: what must it do, what is out of scope.
2. **List interactions**: every click, key, timer and what it should produce.
3. **Sketch the architecture**: which components, who owns which state.
4. **Design the state**: the smallest set of values; derive the rest.
5. **Define data and contracts**: data shape, props in / events out.
6. **Trace one flow**: one interaction end to end.
7. **Find the hard parts**: name them before you code, so you budget time.
8. **Build in layers**: smallest working version first, then add one layer at a time, testing each.
9. **Cover states and edge cases**: empty, one item, error, loading.
10. **Wrap up**: what you handled, trade-offs, what you would add next.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Interactions, architecture, state, contracts	8 min
Basic carousel (one image, prev / next, wrap)	10 min
Sliding track + dots	10 min
Autoplay + pause on hover / focus	10 min
Keyboard, swipe, a11y, edge cases (empty, 1 image, broken image)	12 min
Test the checklist, wrap-up	5 min

> **Think out loud.** The interviewer grades how you decide, not just the final code. Say "I am storing only the index because everything else can be derived", "I am using a functional update because the timer closure may be stale". Silent coding looks like guessing.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show one image at a time from a list of images.
- **Next / Previous** buttons move one slide; they **wrap** (last → first, first → last).
- **Dot indicators** show the current slide and jump to any slide on click.
- **Autoplay**: move to the next slide every N ms (default 3000).
- **Pause** autoplay while the user hovers or focuses the carousel.
- Smooth **slide transition** between images.

Non-functional requirements

Area	What it means here
Performance	Only the transform changes on navigation (GPU friendly), no re-mounting images, one timer at a time.
Scalability	Works for 1..N images; images come from props, not hard-coded.
Error handling	Broken image URL shows a fallback, empty list shows a message, never crashes.
Accessibility	Buttons have labels, arrow keys work, `aria-current` on the active dot, autoplay pauses on focus, respects `prefers-reduced-motion`.
Usability	Swipe on touch devices, manual click restarts the autoplay countdown.
Reusability	`<ImageCarousel images interval autoPlay />`: no knowledge of where data comes from.
Extensibility	Easy to add thumbnails, captions, infinite loop, lazy loading later.

Clarifying questions to ask

1. Should navigation **wrap around** at the ends, or stop (disable the button)?
2. Is **autoplay** required? What interval? Should it pause on hover / focus?
3. Transition type: **slide** or **fade**? Any animation duration constraint?
4. Where do images come from: a static list or an **API**? Do they have alt text?
5. Do we need **touch swipe** and **keyboard** support?
6. Any limit on the number of images (5 vs 500)? Do we need lazy loading?

Assumptions we build with

- Images are passed as props: `{ id, src, alt }[]`, static, a handful (≤ 20).
- Navigation wraps in both directions.
- Autoplay is on by default, 3000 ms, pauses on hover and on keyboard focus.
- Slide transition (horizontal), 700 ms, disabled when the user prefers reduced motion.
- Keyboard (← / →) and touch swipe are in scope. Thumbnails, captions, fullscreen are out of scope.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads with 4 images	First image visible, dot 1 active, autoplay timer starts
Click **Next**	Slides to the next image, active dot moves, autoplay countdown restarts
Click **Next** on the last image	Wraps to the first image
Click **Prev** on the first image	Wraps to the last image
Click dot 3	Jumps straight to image 3
Wait 3 s (no hover)	Advances to the next image automatically
Mouse enters the carousel	Autoplay pauses
Mouse leaves	Autoplay resumes with a fresh 3 s countdown
Tab into the carousel	Focus ring visible, autoplay pauses
Press ← / → while focused	Previous / next slide
Swipe left / right (touch, > 50 px)	Next / previous slide; a tiny drag does nothing
Image URL fails to load	That slide shows "Image failed to load", others work
`images = []`	Shows "No images to show.", no buttons, no timer
`images` has 1 item	Shows the image, no buttons / dots, no autoplay
User prefers reduced motion	Slides change instantly (no animation)
Component unmounts	Timer is cleared, no state update after unmount

This table is the implementation checklist and the manual test plan.

3. Design the architecture

Module	Responsibility
`App`	Page layout; passes the image list and config to the carousel.
`ImageCarousel`	Owns the current index, autoplay, pause, input handling (click, key, swipe), renders track + controls.
`utils/carousel.js`	Pure helper `getNextIndex(index, length, dir)` : the wrap-around math, unit-testable.
`data/images.js`	Static image list with stable ids (would be an API call in real life).

```
App
└── ImageCarousel  (state: currentIndex, isPaused, failedIds)
    ├── Track (div, translateX)
```

```
    │   └── Slide × N  (img or fallback)
    ├── Prev button / Next button
    └── Dots × N
```

Start with a single component. Track, buttons and dots are a few lines each, so splitting them into `Slide`, `Dots`, `Arrow` components would add props plumbing with no gain. Split only when they grow (e.g. thumbnails with their own scroll logic) or get reused.

---

4. Design the state / data model

State or field	Purpose
`currentIndex` (state)	Which slide is shown. The only "real" navigation state.
`isPaused` (state)	True while hovered or focused; stops autoplay.
`failedIds` (state, `Set`)	Ids of images that failed to load, to render a fallback.
`touchStartX` (ref)	X position where a touch began; a ref because changing it must not re-render.
`images`, `interval`, `autoPlay` (props)	Data and config from the parent.

```
const [currentIndex, setCurrentIndex] = useState(0);
const [isPaused, setIsPaused] = useState(false);
const [failedIds, setFailedIds] = useState(() => new Set());
const touchStartX = useRef(null);

const count = images.length;
const index = count ? Math.min(currentIndex, count - 1) : 0; // derived, safe
```

Senior note

- **Derived, not stored:** the transform (`-index * 100%`), which dot is active, "is first / is last", whether controls show (`count > 1`), the `aria-live` value. Storing any of these would create two sources of truth that can drift.
- **Clamped index is derived too:** if the parent passes a shorter list, `Math.min(currentIndex, count - 1)` keeps us in range without a "sync state" effect.
- **Ownership:** the carousel owns its index (uncontrolled). If the parent ever needs it (e.g. sync with a thumbnail strip), lift it up and make the component controlled with `index` + `onChange` props.
- **Ref vs state:** `touchStartX` never affects what is rendered, so it is a ref.

---

5. Design the data structures and contracts

Example data

```
export const IMAGES = [
  { id: "work",    src: "https://cdn.pixabay.com/.../work-5382501_1280.jpg",    alt: "Laptop on a work desk" },
  { id: "blur",    src: "https://cdn.pixabay.com/.../blur-1853262_1280.jpg",    alt: "Blurred desk with a notebook" },
  { id: "desktop", src: "https://cdn.pixabay.com/.../desktop-1985856_1280.jpg", alt: "Desktop setup with a keyboard" },
  { id: "setup",   src: "https://cdn.pixabay.com/.../desktop-5636733_1280.jpg", alt: "Minimal desktop workspace" },
];
```

Why objects, not plain URL strings:

- **Stable** `id` is the React key and the key in `failedIds`. Using the index as key breaks when images are reordered; using the URL breaks when the same image appears twice.
- `alt` is required for screen readers; a URL cannot describe the picture.

Component contract

```
<ImageCarousel
  images={[{ id, src, alt }]}   // required, 0..N items
  interval={3000}               // ms between auto slides, optional
  autoPlay={true}               // optional
/>
```

Input	Output
`images = []`	Empty message
`images.length === 1`	Single image, no controls
`images.length > 1`	Full carousel

If it came from an API (mock contract)

```
GET /api/banners
200 OK
{ "items": [ { "id": "work", "url": "https://...", "altText": "Laptop on a work desk" } ] }
```

Map it once at the boundary: `items.map(({ id, url, altText }) => ({ id, src: url, alt: altText }))`, so the component keeps its own simple shape.

---

6. Trace the data flow

Interaction: user clicks Next on the last slide while autoplay is running.

```
Click "Next" (index 3, count 4)
   │
   ▼
go(1) → setCurrentIndex(i => getNextIndex(3, 4, 1))
   │        (3 + 1 + 4) % 4 = 0
   ▼
State: currentIndex = 0
   │
   ▼
Re-render: index = 0
   ├─ track style: translateX(-0%)  → CSS transition slides 700 ms
   ├─ dot 1 gets aria-current="true", wide white pill
   └─ slide 1 aria-hidden=false, others true
   │
   ▼
Effect deps changed (index 3 → 0)
   ├─ cleanup: clearTimeout(old timer)
   └─ new setTimeout(next, 3000)   ← countdown restarts from the click
   │
   ▼
Output: first image visible, autoplay continues 3 s later
```

---

7. Important technical challenges

Challenge 1: Wrap-around index without off-by-one bugs

Problem: writing `if (i === len - 1) 0 else i + 1` for next and a different branch for prev duplicates logic and is easy to get wrong (`i - 1` gives `-1` on the first slide → blank screen).

Solution: one pure function using modulo. Adding `length` before `%` keeps the result positive for `dir = -1`. Pure → testable with plain asserts.

```
export const getNextIndex = (index, length, dir) =>
  (index + dir + length) % length;

getNextIndex(3, 4, 1);  // 0  (wrap forward)
getNextIndex(0, 4, -1); // 3  (wrap backward)
```

Challenge 2: Autoplay timer that doesn't drift, leak or go stale

Problem: scenario: `setInterval` created once in `useEffect(() => ..., [])` captures `currentIndex = 0` forever (stale closure), so it keeps setting index 1. Also: user clicks Next at 2.9 s, then autoplay fires 0.1 s later: two slides jump almost together. And without cleanup the timer keeps running after unmount.

Solution:

- Use a **functional update** (`setCurrentIndex(i => ...)`) so the timer never reads a stale index.
- Use `setTimeout` **per slide** with `index` in the deps: every slide change (auto or manual) clears the old timer and starts a fresh countdown.
- Return `clearTimeout` from the effect: covers unmount, pause and config changes.
- Skip the timer when paused, autoplay is off, or there are fewer than 2 slides.

```
useEffect(() => {
  if (!autoPlay || isPaused || count < 2) return;
  const id = setTimeout(() => setCurrentIndex((i) => getNextIndex(i, count, 1)), interval);
  return () => clearTimeout(id);
}, [index, isPaused, autoPlay, interval, count]);
```

Challenge 3: Smooth slide animation

Problem: swapping `src` on a single `<img>` cannot animate between two images (the old one is gone instantly), and a new image may flash white while it downloads.

Solution: render **all slides in a flex row** inside an `overflow: hidden` frame and move the row with `transform: translateX(-index * 100%)` plus a CSS transition. Transforms are composited on the GPU (no layout), images stay mounted and decoded, and `motion-reduce:transition-none` respects user settings.

```
<div
  className="flex h-full transition-transform duration-700 ease-in-out motion-reduce:transition-none"
  style={{ transform: `translateX(-${index * 100}%)` }}
>
  {images.map((img) => (
    <div key={img.id} className="w-full h-full shrink-0">…</div>
  ))}
</div>
```

Challenge 4: Accessible pause, keyboard and swipe

Problem: auto-moving content is a WCAG issue (2.2.2 Pause, Stop, Hide): a keyboard user tabbing to "Next" sees the slide change under them; a screen reader announcing every auto slide is noise. Mobile users expect swipe.

Solution:

- Pause on `mouseenter` **and** `focus` ; resume on `blur` only when focus leaves the whole carousel (`relatedTarget` check), not when it moves between its own buttons.
- `aria-live="off"` while autoplaying, `"polite"` when paused, so only user-driven changes are announced.
- Arrow keys on the focusable `section` ; swipe with a threshold so taps don't navigate.

```
onFocus={() => setIsPaused(true)}
onBlur={(e) => !e.currentTarget.contains(e.relatedTarget) && setIsPaused(false)}

const onTouchEnd = (e) => {
  const dx = e.changedTouches[0].clientX - touchStartX.current;
  if (Math.abs(dx) > SWIPE_THRESHOLD) go(dx < 0 ? 1 : -1);
};
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	Static data file with `{ id, src, alt }`	Data ready, stable keys
2	`ImageCarousel` shows `images[0]` in a fixed-size, rounded frame	One image on screen
3	`currentIndex` state + Prev / Next buttons (no wrap)	Manual navigation
4	`getNextIndex` with modulo + node asserts	Wraps both ways, tested
5	Replace single `<img>` with flex track + `translateX` + transition	Smooth sliding
6	Dots with `aria-current`, click to jump	Position indicator
7	Autoplay `useEffect` with `setTimeout` + cleanup	Auto slides, manual click resets countdown
8	`isPaused` via hover / focus / blur	Pauses correctly
9	Keyboard arrows + `tabIndex`, labels, `aria-live`, focus ring	Accessible
10	Touch swipe with threshold	Mobile friendly
11	Edge cases: empty list, 1 image, `onError` fallback, clamp index	Never crashes
12	Walk the interaction table from section 2	Verified

---

9. Cover all states and edge cases

State / edge case	How it is handled
Initial	Index 0, dot 1 active, timer starts
Loading image	Slide has a grey background until the image paints
Success	Image shown with `object-cover`
Empty (`[]`)	"No images to show." No buttons, dots or timer
Single image	Image only; controls hidden; effect returns early (`count < 2`)

State / edge case	How it is handled
Error (broken URL)	`onError` adds the id to `failedIds` → "Image failed to load" fallback for that slide
Paused (hover / focus)	Effect returns early, no timer
Resumed	Fresh full interval, not the remainder
Last → Next / First → Prev	Modulo wraps
List shrinks while on a high index	`Math.min(currentIndex, count - 1)` clamps
Rapid clicks	Each click is a functional update; transition just retargets
Tiny touch drag (< 50 px)	Ignored, treated as a tap
Focus moves between inner buttons	`relatedTarget` check keeps it paused
Reduced motion	`motion-reduce:transition-none`
Unmount	Effect cleanup clears the timer
Background tab	Browsers throttle timers; no extra work needed for this scope

---

10. Final structure

```
_20_image_carousel/
├── index.html
├── package.json
├── vite.config.js            (Tailwind plugin)
└── src/
    ├── main.jsx
    ├── index.css             (@import "tailwindcss")
    ├── App.css
    ├── App.jsx               page layout, passes images
    ├── data/
    │   └── images.js         { id, src, alt }[]
    ├── utils/
    │   └── carousel.js       getNextIndex, SWIPE_THRESHOLD (pure)
    └── components/
        └── ImageCarousel.jsx state, autoplay, input, render
check.mjs                     node assert self-check for getNextIndex
```

---

11. Full working code

`src/utils/carousel.js`

```
// Wrap-around index: dir = +1 (next) or -1 (prev). Works for any length >= 1.
export const getNextIndex = (index, length, dir) =>
  (index + dir + length) % length;

// Minimum horizontal drag (px) that counts as a swipe.
export const SWIPE_THRESHOLD = 50;
```

src/data/images.js

```
export const IMAGES = [
  { id: "work", src: "https://cdn.pixabay.com/photo/2020/07/08/04/12/work-5382501_1280.jpg", alt:
"Laptop on a work desk" },
  { id: "blur", src: "https://cdn.pixabay.com/photo/2016/11/23/14/37/blur-1853262_1280.jpg", alt:
"Blurred desk with a notebook" },
  { id: "desktop", src: "https://cdn.pixabay.com/photo/2017/01/17/03/57/desktop-1985856_1280.jpg", alt:
"Desktop setup with a keyboard" },
  { id: "setup", src: "https://cdn.pixabay.com/photo/2020/10/08/02/41/desktop-5636733_1280.jpg", alt:
"Minimal desktop workspace" },
];
```

src/components/ImageCarousel.jsx

```
import { useEffect, useRef, useState } from "react";
import { GrPrevious, GrNext } from "react-icons/gr";
import { getNextIndex, SWIPE_THRESHOLD } from "../utils/carousel";

const ImageCarousel = ({ images = [], interval = 3000, autoPlay = true }) => {
  const [currentIndex, setCurrentIndex] = useState(0);
  const [isPaused, setIsPaused] = useState(false);
  const [failedIds, setFailedIds] = useState(() => new Set());
  const touchStartX = useRef(null);

  const count = images.length;
  // Derived: if the list shrinks, never point past the end.
  const index = count ? Math.min(currentIndex, count - 1) : 0;

  const go = (dir) => setCurrentIndex((i) => getNextIndex(Math.min(i, count - 1), count, dir));
  const goTo = (i) => setCurrentIndex(i);

  // Autoplay: a fresh timeout per slide, so manual navigation restarts the countdown.
  useEffect(() => {
    if (!autoPlay || isPaused || count < 2) return;
    const id = setTimeout(() => setCurrentIndex((i) => getNextIndex(i, count, 1)), interval);
    return () => clearTimeout(id);
  }, [index, isPaused, autoPlay, interval, count]);
```

```
  const onKeyDown = (e) => {
    if (e.key === "ArrowLeft") go(-1);
    if (e.key === "ArrowRight") go(1);
  };

  const onTouchStart = (e) => { touchStartX.current = e.touches[0].clientX; };
  const onTouchEnd = (e) => {
    if (touchStartX.current === null) return;
    const dx = e.changedTouches[0].clientX - touchStartX.current;
    if (Math.abs(dx) > SWIPE_THRESHOLD) go(dx < 0 ? 1 : -1);
    touchStartX.current = null;
  };

  if (count === 0) {
    return <p className="text-gray-500">No images to show.</p>;
  }

  return (
    <section
      aria-roledescription="carousel"
      aria-label="Image carousel"
      tabIndex={0}
      onKeyDown={onKeyDown}
      onMouseEnter={() => setIsPaused(true)}
      onMouseLeave={() => setIsPaused(false)}
      onFocus={() => setIsPaused(true)}
      onBlur={(e) => !e.currentTarget.contains(e.relatedTarget) && setIsPaused(false)}
      onTouchStart={onTouchStart}
      onTouchEnd={onTouchEnd}
      className="relative w-[90vw] max-w-[600px] aspect-[3/2] overflow-hidden rounded-2xl shadow-2xl
                 outline-none focus-visible:ring-4 focus-visible:ring-blue-400"
    >
      {/* Track: all slides side by side, moved with translateX */}
      <div
        className="flex h-full transition-transform duration-700 ease-in-out motion-reduce:transition-
none"
        style={{ transform: `translateX(-${index * 100}%)` }}
        aria-live={autoPlay && !isPaused ? "off" : "polite"}
      >
        {images.map((img, i) => (
          <div
            key={img.id}
            role="group"
            aria-roledescription="slide"
            aria-label={`${i + 1} of ${count}`}
            aria-hidden={i !== index}
```

```
            className="w-full h-full shrink-0 bg-gray-200"
          >
            {failedIds.has(img.id) ? (
              <div className="w-full h-full flex items-center justify-center text-gray-500">
                Image failed to load
              </div>
            ) : (
              <img
                src={img.src}
                alt={img.alt}
                draggable={false}
                onError={() => setFailedIds((s) => new Set(s).add(img.id))}
                className="w-full h-full object-cover"
              />
            )}
          </div>
        ))}
      </div>

      {count > 1 && (
        <>
          <button
            type="button"
            aria-label="Previous slide"
            onClick={() => go(-1)}
            className="absolute top-1/2 -translate-y-1/2 left-4 p-3 rounded-full bg-white/70 hover:bg-
blue-500 hover:text-white cursor-pointer"
          >
            <GrPrevious />
          </button>
          <button
            type="button"
            aria-label="Next slide"
            onClick={() => go(1)}
            className="absolute top-1/2 -translate-y-1/2 right-4 p-3 rounded-full bg-white/70 hover:bg-
blue-500 hover:text-white cursor-pointer"
          >
            <GrNext />
          </button>

          <div className="absolute bottom-3 left-1/2 -translate-x-1/2 flex gap-2">
            {images.map((img, i) => (
              <button
                key={img.id}
                type="button"
                aria-label={`Go to slide ${i + 1}`}
                aria-current={i === index}
```

```
                onClick={() => goTo(i)}
                className={`h-3 rounded-full transition-all cursor-pointer ${
                  i === index ? "w-6 bg-white" : "w-3 bg-white/50"
                }`}
              />
            ))}
          </div>
        </>
      )}
    </section>
  );
};

export default ImageCarousel;
```

`src/App.jsx`

```
import "./App.css";
import ImageCarousel from "./components/ImageCarousel";
import { IMAGES } from "./data/images";

const App = () => (
  <div className="flex p-4 mt-36 justify-center items-center">
    <ImageCarousel images={IMAGES} interval={3000} />
  </div>
);

export default App;
```

`check.mjs` (run with `node check.mjs`)

```
import assert from "node:assert";
import { getNextIndex } from "./src/utils/carousel.js";

assert.equal(getNextIndex(0, 4, 1), 1);
assert.equal(getNextIndex(3, 4, 1), 0); // wrap forward
assert.equal(getNextIndex(0, 4, -1), 3); // wrap backward
assert.equal(getNextIndex(2, 4, -1), 1);
assert.equal(getNextIndex(0, 1, 1), 0); // single slide stays
assert.equal(getNextIndex(0, 1, -1), 0);
console.log("self-check ok");
```

`main.jsx`, `index.css` (`@import "tailwindcss";`) and `vite.config.js` stay as generated by the template.

---

12. Wrap-up

Edge cases handled

Empty list, single image, broken image URL, wrap in both directions, list shrinking under a high index, rapid clicks, pause on hover and focus (with inner focus moves), tiny swipes, reduced motion, timer cleanup on unmount.

Trade-offs to mention

- **All slides mounted** → smooth transitions and no flash, but every image downloads up front. Fine for a handful; for many images render only current ± 1 or lazy-load.
- `setTimeout` **per slide** instead of `setInterval` → manual navigation resets the countdown and there is no stale closure, at the cost of creating a timer per slide (negligible).
- **Resume restarts the full interval** rather than the remaining time: simpler, and users rarely notice.
- **Wrap with a rewind animation** (last → first slides back across all images) instead of a true infinite loop with cloned slides: much simpler, slightly less polished.
- **Uncontrolled component**: easy to drop in; lift state up if the parent needs the index.
- **Single component** instead of `Slide` / `Dots` / `Arrow`: fewer props to pass for this size.

What to add with more time

- Tests: React Testing Library + fake timers (`vi.useFakeTimers()`) for autoplay, pause, wrap, keyboard.
- Lazy loading / `srcset` + `sizes` for responsive images; preload next image only.
- Infinite loop with cloned first / last slides and a no-transition jump.
- Visible pause / play button (WCAG requires a way to stop motion, hover isn't available on touch).
- Pause when the tab is hidden (`visibilitychange`) or the carousel is off screen (`IntersectionObserver`).
- Captions, thumbnails, fullscreen, controlled mode (`index` + `onChange`).

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you ask whether navigation should wrap?

It changes the UI and the logic: wrapping needs modulo math, not wrapping needs disabled buttons at the ends and the autoplay must stop or rewind. Asking up front avoids rewriting the navigation later.

Q2. Is autoplay a good idea at all?

It hurts accessibility and attention, so WCAG 2.2.2 requires a way to pause. I pause on hover and focus and keep it optional through the `autoPlay` prop; with more time I would add a visible pause button.

State design

Q3. Why store only the index and not the current image object?

The image is `images[index]`: derived. Storing it too means two sources of truth; if the list changes the stored object can point to a removed image.

Q4. Why is `touchStartX` a ref and not state?

It never affects the rendered output. State would trigger a re-render on every touch start for nothing. A ref keeps the value between renders without re-rendering.

Q5. What happens if the parent passes a shorter list while we are on slide 4?

`index = Math.min(currentIndex, count - 1)` clamps it during render, so we never read `images[4]` of a 2-item list. No `useEffect` "sync" needed, which would render once with a bad index first.

Challenge: wrap-around

Q6. Why `(index + dir + length) % length` and not just `% length`?

In JavaScript `-1 % 4` is `-1`, not `3`. Adding `length` first keeps the value non-negative for `dir = -1`. It works because `dir` is never smaller than `-length`.

Q7. How would you build a true infinite loop with no rewind animation?

Clone the last slide before the first and the first after the last. Slide into the clone with a transition, then on `transitionend` jump to the real slide with the transition turned off. Index math moves to the 1..N range.

Challenge: autoplay timer

Q8. Why not `setInterval` in a `useEffect` with an empty dependency array?

The callback would capture the first render's values (stale closure) and the interval would not reset after a manual click, so the next auto slide could fire 100 ms after the user clicked. `setTimeout` keyed on `index` restarts on every slide change.

Q9. Why the functional update `setCurrentIndex(i => ...)` inside the timer?

The timer runs after the render that created it; the functional form always gets the latest state, so even a delayed or doubled call cannot read a stale index.

Q10. In React StrictMode effects run twice in dev. Will we get two timers?

No. StrictMode runs setup → cleanup → setup, and our cleanup clears the first timeout. Only one timer is live. That double run is exactly what catches a missing cleanup.

Q11. What if the user switches tabs?

Browsers throttle timers in background tabs, so it slows down on its own. To stop it fully, listen to `visibilitychange` and set `isPaused` when `document.hidden` is true.

Challenge: animation and performance

Q12. Why `transform: translateX` instead of changing `left` or `margin-left`?

`left` / `margin` trigger layout on every frame; `transform` is handled by the compositor on the GPU, so it stays at 60 fps and does not move the surrounding page.

Q13. What if there are 500 images?

Mounting 500 `<img>` downloads all of them. Render only a window (current, previous, next) positioned absolutely, or keep the track and add `loading="lazy"` with real dimensions, plus `srcset` for size. Dots would become a counter ("12 / 500") or a scrollable thumbnail strip.

Q14. What if the image API is slow?

Show a skeleton or the grey slide background while the list loads, a fixed aspect-ratio frame so the layout does not jump, and an error state with retry if the request fails. Preload the next image only (`new Image().src = next.src`) so the following slide is ready.

Q15. Why does a parent re-render re-render the carousel, and should you memo it?

Parent renders re-render children by default. Here the carousel is cheap (a few nodes, transform change), so `React.memo` adds comparison cost for little gain. If the parent re-renders often and passes a stable `images` array, `memo` is fine; an inline array literal would defeat it anyway.

Implementation steps

Q16. Why use `button` elements for dots instead of `span` with `onClick`?

Buttons are focusable, work with Enter / Space, and are announced as buttons. A clickable `span` needs `tabIndex` , `role` and key handlers just to reach the same baseline.

Q17. Why check `relatedTarget` in `onBlur`?

React's `onBlur` bubbles: moving focus from the Prev button to the Next button blurs one child. Without the check autoplay would resume for a moment between them. `contains(relatedTarget)` resumes only when focus leaves the carousel.

Q18. How would you test the autoplay?

React Testing Library with fake timers: render, `vi.advanceTimersByTime(3000)` , expect dot 2 to have `aria-current="true"` ; hover, advance again, expect it unchanged; unmount and check no warnings. The wrap math is already covered by plain asserts.

Trade-offs and "why not X?"

Q19. Why not use a library like Swiper or react-slick?

In production, often yes: they handle infinite loop, touch physics and a11y. In an interview the point is to show the state, timer and animation reasoning, and the native version is about 120 lines with no dependency.

Q20. Why not CSS scroll-snap instead of translateX?

Scroll snap gives native swipe and momentum for free and is a good choice. Programmatic control (autoplay, dots, wrap) then goes through `scrollTo` and syncing the index from scroll events, which is more code and harder to keep in sync. For a timer-driven carousel, a state-driven transform is simpler.

Q21. Why a single component and not `Slide`, `Dots`, `Arrows`?

Each part is a few lines and depends on the same index and handlers. Splitting would mean passing `index`, `count`, `goTo` everywhere. I would split when a piece grows its own logic (e.g. a scrolling thumbnail strip) or is reused.

Q22. How would you make it controlled?

Accept `index` and `onChange` props. When `index` is passed, use it instead of internal state and call `onChange(next)` instead of `setCurrentIndex`. That lets a parent sync it with thumbnails or the URL (`?slide=3`).